

DATA SCIENCE PROJECT REPORT

Income / Census Dataset

Exploratory Data Analysis

Data Cleaning · Feature Engineering · Salary Prediction Insights · Memory Optimization

Prince Maheta

Data Science | Aspiring Analyst

GitHub: [princemaheta2627-glitch](#)

June 2025

Executive Summary

This report documents an Exploratory Data Analysis (EDA) on the UCI Adult (Census Income) dataset — one of the most widely studied classification datasets in machine learning. Containing 48,842 records of demographic and employment attributes, the goal is to explore the data's structure, clean it rigorously, and derive insights about factors that influence whether an individual earns more or less than \$50,000 per year.

What distinguishes this project from basic EDA is its comprehensive focus on data engineering skills: handling disguised missing values ('?'), dropping irrelevant features, removing duplicates, label encoding the target variable, dtype conversion for memory optimization, and applying both univariate and bivariate analysis techniques — all critical competencies for real-world data preparation pipelines.

Project Highlights

Metric	Value
Dataset Size	48,842 rows × 15 columns (raw); 45,175 × 12 (cleaned)
Target Variable	income: ≤50K (0) or >50K (1)
High-Income Population	11,687 records (23.9%) earn >\$50K

Metric	Value
Low-Income Population	37,155 records (76.1%) earn ≤\$50K
Missing Values ('?')	workclass: 2,799 occupation: 2,809 native-country: 857
Rows after Cleaning	45,175 (after null drop + deduplication)
Columns Dropped	educational-num, capital-gain, capital-loss
Top Earning Workclass	Self-Employed-Inc — 55.3% earn >\$50K
Gender Salary Gap	Male: 30.4% earn >\$50K vs Female: 10.9%
Persons aged 17–48	37,321 (76.4% of total)
Bachelors + Masters holders	10,682 persons
Tools Used	Python, Pandas, NumPy, Seaborn, Matplotlib
Key Pandas Functions	isin, between, unique, dropna, replace, drop_duplicates, astype, apply

1. Introduction

1.1 Project Background

The Adult Census Income dataset, originally from the UCI Machine Learning Repository, is extracted from the 1994 US Census database. It is widely used in data science education as a binary classification benchmark — predicting whether an individual's annual income exceeds \$50,000 based on demographic features.

This EDA project goes beyond exploration. It serves as a guided tour through the complete data preparation lifecycle: ingesting messy real-world data, discovering and resolving hidden missing values, removing redundant features, encoding the target variable, and producing a model-ready dataset — all while extracting meaningful demographic and economic insights.

1.2 Learning Goals

This project was specifically designed to teach and demonstrate the following Pandas functions:

Function / Method	Purpose	Applied In
data.sample(frac=0.50)	Fetch a random 50% sample of rows	Q5 — Random Sampling
.isin([...])	Filter rows matching a list of values	Q7 — '?' detection; Q15 — degree filter
.between(low, high)	Filter numeric range (inclusive)	Q13 — age 17 to 48
.unique()	Get distinct values of a column	Q17 — income categories
.dropna()	Remove rows with null/NaN values	Q8 — null cleanup

Function / Method	Purpose	Applied In
<code>.replace(from, to)</code>	Replace specific values in a column	Q7 — '?' → NaN; Q17 — label encode
<code>.duplicated()</code>	Detect duplicate rows	Q9 — check duplication
<code>.drop_duplicates()</code>	Remove duplicate rows	Q9 — deduplication
<code>.astype('category')</code>	Convert dtype to memory-efficient category	Q20 — memory optimization
<code>.apply(func)</code>	Apply a custom function to a column	Referenced in learning goals

1.3 The 20 Questions Answered

#	Question	Category
1	Display top 10 rows of dataset	Inspection
2	Display bottom 10 rows	Inspection
3	Find shape (rows & columns)	Inspection
4	Info: dtypes, null counts, memory	Profiling
5	Fetch 50% random sample	Sampling
6	Check null values	Quality Check
7	Replace '?' with NaN (data cleaning)	Cleaning
8	Drop all missing value rows	Cleaning
9	Check & remove duplicate rows	Cleaning
10	Overall statistics (describe)	Statistics
11	Drop columns: educational-num, capital-gain, capital-loss	Feature Engineering
12	Age distribution (univariate)	Univariate Analysis
13	Persons aged 17–48 using <code>.between()</code>	Filtering
14	Workclass distribution	Univariate Analysis
15	Count Bachelors & Masters holders using <code>.isin()</code>	Filtering
16	Income vs. age boxplot (bivariate)	Bivariate Analysis
17	Label encode income: $\leq 50K \rightarrow 0$, $> 50K \rightarrow 1$	Feature Engineering
18	Which workclass earns highest salary?	Bivariate / Aggregation
19	Gender salary gap: Male vs Female	Bivariate Analysis
20	Convert workclass to category dtype	Memory Optimization

2. Dataset Overview

2.1 Data Source

The dataset is the UCI Adult / Census Income dataset available on Kaggle. It was extracted from the 1994 US Census Bureau database and is maintained by Ronny Kohavi and Barry Becker. It represents real demographic and employment records, making it one of the most realistic beginner datasets available for classification and EDA practice.

Repository: github.com/princemaheta2627-glitch/income_statement_eda

2.2 Raw Dataset Structure

The raw dataset contains 48,842 rows and 15 columns:

Column	Dtype	Description	Notes
age	int64	Person's age (17–90)	Clean, no nulls
workclass	object	Employment type (Private, Gov, Self-emp...)	Contains '?' for missing — 2,799 rows
fnlwgt	int64	Census sampling weight	Dropped implicitly in analysis
education	object	Highest education level (16 categories)	Clean; HS-grad is most common
educational-num	int64	Numeric encoding of education	DROPPED — redundant with education
marital-status	object	Marital status (7 categories)	Clean, no nulls
occupation	object	Job occupation (14 categories)	Contains '?' — 2,809 rows
relationship	object	Family role (Wife, Husband, etc.)	Clean, no nulls
race	object	Race (5 categories)	Clean, no nulls
gender	object	Male / Female	Clean, no nulls
capital-gain	int64	Capital gains income	DROPPED — mostly zero, low signal
capital-loss	int64	Capital losses	DROPPED — mostly zero, low signal
hours-per-week	int64	Weekly working hours	Clean, no nulls
native-country	object	Country of origin (41 countries)	Contains '?' — 857 rows
income	object	Target: <=50K or >50K	Clean; label-encoded to 0/1

2.3 Post-Cleaning Dataset

Stage	Rows	Columns	Action Taken
Raw Load	48,842	15	Initial CSV load
After '?' → NaN Replacement	48,842	15	isin + replace; nulls now visible
After dropna()	45,222	15	Removed 3,620 rows with any NaN
After drop_duplicates()	45,175	15	Removed 47 duplicate rows
After column drop	45,175	12	Removed educational-num, capital-gain, capital-loss

3. Methodology & Data Pipeline

3.1 Tools & Libraries

Library	Role	Key Usage
Python 3	Core language	All logic, lambda functions, custom operations
Pandas	Data manipulation	Full pipeline from load to feature engineering
NumPy	Numerical support	np.nan for null replacement
Seaborn	Statistical visualization	boxplot for bivariate, heatmap for null map
Matplotlib	Plot rendering	hist(), figsize, axis configuration

3.2 Data Cleaning Pipeline

This project demonstrates a complete, production-relevant data cleaning pipeline:

Step	Problem	Method	Result
1. Disguised Nulls	Missing data encoded as '?' strings	data.isin(['?']).sum() → replace('?', np.nan)	True nulls now detectable
2. Null Visualization	Visual confirmation of missing pattern	sns.heatmap(data.isnull())	Yellow bands show null locations
3. Missing % Calc	Quantify missingness per column	isnull().sum() * 100 / len(data)	workclass 5.73%, occupation 5.75%
4. Row Removal	Drop incomplete records	dropna(how='any', inplace=True)	48,842 → 45,222 rows
5. Deduplication	Remove exact duplicate rows	.duplicated().any() → drop_duplicates()	45,222 → 45,175 rows

Step	Problem	Method	Result
6. Feature Drop	Remove redundant/low-signal columns	<code>.drop(['educational-num', 'capital-gain', 'capital-loss'])</code>	15 → 12 columns
7. Label Encoding	Convert target to binary numeric	<code>.replace(['<=50K', '>50K'], [0,1])</code>	income: string → 0/1 integer
8. Dtype Optimization	Reduce memory by recasting	<code>astype('category')</code> on workclass	object → category (lower memory)

3.3 Key Code Patterns

Disguised Missing Value Detection & Replacement

```

- data.isin(['?']).sum()          # workclass: 2799, occupation: 2809
- data['workclass'] = data['workclass'].replace('?', np.nan)
- data['occupation'] = data['occupation'].replace('?', np.nan)
- data['native-country'] = data['native-country'].replace('?', np.nan)

```

Null Percentage Calculation

```

- per_missing = data.isnull().sum() * 100 / len(data)
- # workclass: 5.73%, occupation: 5.75%, native-country: 1.75%

```

Range Filtering with .between()

```

- sum(data['age'].between(17, 48)) # → 37,321 persons

```

Multi-Value Filter with .isin()

```

- sum(data['education'].isin(['Bachelors', 'Masters'])) # → 10,682

```

Label Encoding the Target

```

- data.replace(to_replace=['<=50K', '>50K'], value=[0,1], inplace=True)

```

Groupby for Salary Analysis

```

- data.groupby('workclass')['income'].mean().sort_values(ascending=False)
- data.groupby('gender')['income'].mean().sort_values(ascending=False)

```

Memory Optimization

```

- data['workclass'] = data['workclass'].astype('category')
- # dtype changes from object to category – significant RAM savings at scale

```

4. Key Findings & Insights

4.1 Income Distribution

The target variable income is significantly imbalanced — a critical observation for any future machine learning work:

Income Class	Label	Count	Percentage
<=50K (low income)	0	37,155	76.1%
>50K (high income)	1	11,687	23.9%

Insight: The dataset has a roughly 3:1 class imbalance. Any predictive model built on this data would need to account for this using techniques like SMOTE, class weighting, or stratified sampling to avoid biased predictions toward the majority class.

4.2 Age Distribution (Univariate)

The age column (range: 17–90) was analyzed using a histogram:

- Mean age: 38.6 years, Median: 37 years — slightly right-skewed toward older workers
- 25th percentile: 28 years | 75th percentile: 48 years
- 37,321 persons (76.4%) fall in the 17–48 age range
- The histogram shows a right tail — fewer elderly workers but they extend to age 90

Insight: The workforce is concentrated in the 28–48 prime working-age range, consistent with real US Census labor participation patterns.

4.3 Age vs. Income (Bivariate)

A boxplot comparing age distributions across income classes revealed a clear pattern:

- <=50K earners: Younger median age — concentrated in the 20s–30s
- >50K earners: Older median age — concentrated in the 35–55 range
- The interquartile ranges barely overlap — age is a meaningful predictor of income class

Insight: Age is positively correlated with earning >\$50K. Career progression, seniority, and accumulated experience account for higher earnings in older workers. This confirms age would be a valuable feature in a classification model.

4.4 Workclass Distribution & Salary Impact

Workclass	% Earning >\$50K	Rank	Interpretation
Self-emp-inc	55.3%	1st	Incorporated self-employed — highest earners
Federal-gov	39.2%	2nd	Federal jobs offer above-average compensation

Workclass	% Earning >\$50K	Rank	Interpretation
Local-gov	29.6%	3rd	Local government — moderate salary premium
Self-emp-not-inc	27.9%	4th	Unincorporated self-employed — lower than incorporated
State-gov	26.8%	5th	State government — slightly below local
Private	21.8%	6th	Largest group; widest salary variance
Without-pay	9.5%	7th	Unpaid/volunteer work — near zero high income
Never-worked	0.0%	Last	No income by definition

Insight: Self-incorporated business owners are the strongest predictor of high income. Federal government employment offers significant salary premiums over state/local. Private sector workers, despite being the largest group (69.4% of dataset), have the lowest high-income rate among employed workers.

4.5 Gender & Income (Bivariate)

Gender	% Earning >\$50K	Sample Size	Gap vs. Male
Male	30.4%	~32,650	Baseline
Female	10.9%	~16,192	-19.5 percentage points

Insight: Males are nearly 3x more likely to earn >\$50K than females in this dataset (30.4% vs 10.9%). This reflects the documented gender wage gap present in 1994 US Census data. This feature would be among the top predictors in a classification model, though its inclusion raises important fairness and bias considerations in ML applications.

4.6 Education Analysis

10,682 persons hold either a Bachelors or Masters degree — representing 21.9% of the full dataset. Private workclass is the most common employer (top=Private, freq=33,906 out of 48,842). HS-grad is the most common education level (15,784 persons).

- Bachelors + Masters combined: 10,682 persons
- Most common education: HS-grad — 15,784 persons (32.3%)
- Unique education levels: 16 categories from Preschool to Doctorate

Insight: Despite HS-grad being the most common education level, higher education holders (Bachelors/Masters) represent a significant and high-earning segment that would benefit from targeted analysis.

4.7 Feature Relevance: Dropped Columns

Column Dropped	Reason	Data Science Rationale
educational-num	Redundant — same info as education string	Removing correlated features reduces multicollinearity
capital-gain	Mostly zero — low variance, low signal	Near-constant columns add noise, not information
capital-loss	Mostly zero — low variance, low signal	Same as above; reduces dimensionality cleanly

5. Technical Skills Demonstrated

5.1 Advanced Data Cleaning

- Identified disguised missing values encoded as '?' strings using `.isin(['?']).sum()`
- Replaced '?' with `np.nan` across three columns to expose true null pattern
- Visualized missing data with `sns.heatmap(data.isnull())` — yellow bands reveal null locations
- Quantified missingness: `per_missing = isnull().sum() * 100 / len(data)`
- Applied `dropna(how='any')` to remove 3,620 rows with any null value
- Detected duplicates with `.duplicated().any()` → confirmed True → removed 47 rows

5.2 Pandas Filtering Techniques

- `.between(17, 48)` — inclusive range filter on numeric column (37,321 results)
- `.isin(['Bachelors', 'Masters'])` — multi-value categorical filter (10,682 results)
- Boolean indexing with `|` (OR) operator: `filter1 | filter2`
- `.unique()` — retrieve distinct values before encoding
- `.value_counts()` — frequency distribution for categorical analysis

5.3 Feature Engineering

- Dropped 3 columns (educational-num, capital-gain, capital-loss) based on redundancy analysis
- Label-encoded the binary target variable using `.replace(['<=50K', '>50K'], [0,1])`
- Converted workclass from object to category dtype using `.astype('category')` for memory efficiency

5.4 Analytical Methods

- `data.sample(frac=0.50, random_state=100)` — reproducible 50% random sampling
- `data.describe(include='all')` — combined numeric and categorical stats in one call

- `data['age'].hist()` — univariate distribution visualization
- `sns.boxplot(x='income', y='age')` — bivariate age-by-income comparison
- `data.groupby('workclass')['income'].mean().sort_values()` — aggregated salary rate ranking
- `data.groupby('gender')['income'].mean()` — gender-based salary gap quantification

5.5 Memory Optimization

A key advanced concept introduced in this project is dtype-based memory optimization — converting object columns with limited unique values to the category dtype:

- object dtype: stores each string value individually — $O(n)$ memory
- category dtype: stores a lookup table + integer codes — $O(k + n)$ where $k \ll n$
- For workclass (only 9 unique values across 48,842 rows), this yields significant memory savings
- This pattern is essential for working with large datasets where RAM constraints matter

6. Conclusion & Future Work

6.1 Summary of Findings

- 48,842 census records cleaned down to 45,175 usable, deduplicated rows
- Disguised missing values ('?') found in 3 columns — total 6,465 records affected
- 76.1% earn $\leq \$50K$; 23.9% earn $> \$50K$ — significant class imbalance
- Age is positively correlated with high income — older workers earn more
- Self-incorporated entrepreneurs have the highest high-income rate (55.3%)
- A 19.5 percentage-point gender income gap exists — males earn $> \$50K$ at 3x the female rate
- HS-grad is the most common education; Bachelors+Masters = 21.9% of dataset
- workclass successfully converted to category dtype for memory optimization

6.2 Potential Next Steps

Enhancement	Description	Technique
Income Classification Model	Predict $> \$50K$ using cleaned features	Logistic Regression, Random Forest, XGBoost
Fairness Analysis	Measure and mitigate gender/race bias	IBM AI Fairness 360, SHAP values
Class Imbalance Handling	Address 3:1 imbalance in target	SMOTE, <code>class_weight='balanced'</code>

Enhancement	Description	Technique
Feature Importance	Identify top income predictors	Random Forest feature_importances_
Education–Income Deep Dive	Income rates across all 16 education levels	groupby + barplot
Hours vs Income	Does overworking correlate with high income?	Scatter + correlation analysis

6.3 Learning Outcomes

This project was the most technically rigorous in the portfolio series — going well beyond visualization into serious data engineering. The '?' disguised missing value problem is a classic real-world data quality issue, and solving it using `.isin()` + `.replace()` + `.dropna()` builds an intuition for handling similar surprises in production data. The memory optimization section introduced an important concept rarely covered in beginner EDA: thinking about computational efficiency, not just analytical correctness.

7. Project Information

Project Title	Income / Census EDA: Data Cleaning, Encoding & Salary Insights
Author	Prince Maheta
GitHub Repository	github.com/princemaheta2627-glitch/income_statement_eda
Notebook File	3-Income-EDA-Project-answers.ipynb
Dataset	UCI Adult (Census Income) — Kaggle (48,842 rows, 15 columns)
Data Origin	1994 US Census Bureau — Extracted by Ronny Kohavi & Barry Becker
Platform	Kaggle Notebook / Jupyter (Python 3)
Languages & Tools	Python 3, Pandas, NumPy, Seaborn, Matplotlib
Key Pandas Methods	<code>isin</code> , <code>between</code> , <code>unique</code> , <code>dropna</code> , <code>replace</code> , <code>drop_duplicates</code> , <code>astype</code> , <code>apply</code> , <code>sample</code>

Date Completed	June 2025
-----------------------	-----------

Report Date	June 7, 2026
--------------------	--------------

This report was prepared as part of a data science portfolio.
Prince Maheta | Data Science Portfolio | 2025